

Welcome to

8. Programming and Memory Corruption

EK Software Security

Henrik Kramselund he/him han/ham hlk@zencurity.com @kramse

Slides are available as PDF, https://codeberg.org/kramse
8-programming-and-mem-corruption.tex in the repo security-courses

© license CC BY 3.0. 2026 Henrik Kramselund

Goals for today



Todays goals:

- Web Application Hacking – do offensive testing on web applications
- Perform analysis of the underlying factors, why is it possible
- Understand more C, and problems associated with C
- Talk about the Design Patterns concept, and the ones often found in programs
- Patterns can also be found in other areas, like Patterns in Network Architecture

Photo by Thomas Galler on Unsplash

Plan for today



Subjects

- Offense against web applications: XSS, CSRF, XXE, injection, DoS, third-party dependencies
- C language issues, memory corruption errors, buffer overflows, stack errors
- Common constructs and recurring code patterns
- Example programs with flaws: OpenSSH, OpenSSL, Windows MS-RPC DCOM, Linux teardrop

Exercises

- JuiceShop attacks, including SQL injection login
- Writing and exploiting a small buffer overflow, run debugger
- Pointers and Structure padding
- Static analysis with PMD, Git hook example, use a XML library in Python

Time schedule



Plan for the day

12:45 - 13:45 – Teaching: Offense against web applications, XSS, CSRF, XXE, injection
13:45 - 14:00 – 15min break
14:00 - 14:45 – 45min Exercises: JuiceShop attacks, including SQL injection login
14:45 - 15:00 – 15min break
15:00 - 15:45 – 45min Teaching: C issues, memory corruption, program building blocks
15:45 - 16:00 – 15min break
16:00 - 16:45 – 45min Exercises: Buffer overflow and debugger, structure padding

12:45 - 16:45 I will be here at EK

Next up Offensive Web Hacking



Input from this book – you dont need to have it, the slides cover the important parts:
Web Application Security, Andrew Hoffman, 2020, ISBN: 9781492053118

Part II. Offense, chapters 9-16, very short chapters

9. Introduction to Hacking Web Applications
10. Cross-Site Scripting (XSS)
11. Cross-Site Request Forgery (CSRF)
12. XML External Entity (XXE)
13. Injection
14. Denial of Service (DoS)
15. Exploiting Third-Party Dependencies
16. Part II Summary

What we learnt from the recon part



> In Part I of this book, "Recon," we explored a number of ways to investigate and document the structure and function of a web application. We evaluated ways of finding APIs on a server, including those that exist on subdomains rather than at just the top-level domain. We considered methods of enumerating the endpoints that those APIs exposed, and the HTTP verbs that they accepted.

Source: *Web Application Security*, Andrew Hoffman, 2020, ISBN: 9781492053118

- We are allowed to do reconnaissance
- Having a good inventory of software, hardware, devices, etc. makes it easier to control and secure
- Having knowledge about patching levels is vital!

Exploiting web applications is exploiting software



> In the next few chapters, you will learn how to take advantage of web applications through a number of powerful and common exploitation techniques. As you learn about these techniques, consider the lessons from the previous part and attempt to brainstorm how those recon techniques would be useful in helping you find weaknesses in an application where the upcoming exploits you'll learn about be applied.

Source: *Web Application Security*, Andrew Hoffman, 2020, ISBN: 9781492053118

- Attacking web applications attack the whole organisation
- Web applications are exposing the organisation
- Vulnerabilities in the web applications typically allow access to data/databases



9. Introduction to Hacking Web Applications

> Software engineers measure productivity in value-add through features, or improvements to an existing codebase. A software engineer might say, "I added features x and y, hence today was a good day." Alternatively, they might say, "I improved the performance of features a and b by 10%," alluding to the fact that the work of a software engineer, while difficult to measure compared to traditional occupations, is still quantifiably measurable.
>
> Hackers measure productivity in ways that are much more difficult to discern and measure. This is because the majority of hacking is actually data gathering and analysis. Often this process is riddled with false positives and might look like time wasted to an uneducated onlooker.

Source: *Web Application Security*, Andrew Hoffman, 2020, ISBN: 9781492053118

10. Cross-Site Scripting (XSS)



> Cross-Site Scripting (XSS) vulnerabilities are some of the most common vulnerabilities throughout the internet, and have appeared as a direct response to the increasing amount of user interaction in today's web applications. At its core, an XSS attack functions by taking advantage of the fact that web applications execute scripts on users' browsers. Any type of dynamically created script that is executed puts a web application at risk if the script being executed can be contaminated or modified in any way—in particular by an end user.

XSS attacks are categorized a number of ways, with the big three being:

- Stored (the code is stored on a database prior to execution)
- Reflected (the code is not stored in a database, but reflected by a server)
- DOM-based (code is both stored and executed in the browser)

Note: attacks the user, his/her data mostly.

11. Cross-Site Request Forgery (CSRF)



Sometimes we already know an API endpoint exists that would allow us to perform an operation we wish to perform, but we do not have access to that endpoint because it requires privileged access (e.g., an admin account).

In this chapter, we will be building Cross-Site Request Forgery (CSRF) exploits that result in an admin or privileged account performing an operation on our behalf rather than using a JavaScript code snippet.

CSRF attacks take advantage of the way browsers operate and the trust relationship between a website and the browser. By finding API calls that rely on this relationship to ensure security—but yield too much trust to the browser—we can craft links and forms that with a little bit of effort can cause a user to make requests on his or her own behalf—unknown to the user generating the request.

- Often seen in small CPE routers in homes, if the user activates and evil link, their router might be reconfigured or taken over

12. XML External Entity (XXE)



XML External Entity (XXE) is a classification of attack that is often very simple to execute, but with devastating results. This classification of attack relies on an improperly configured XML parser within an application's code.

Generally speaking, almost all XXE attack vulnerabilities are found as a result of an API endpoint that accepts an XML (or XML-like) payload. You may think that HTTP endpoints accepting XML is uncommon, but XML-like formats include SVG, HTML/DOM, PDF (XFDF), and RTF. These XML-like formats share many common similarities with the XML spec, and as result, many XML parsers also accept them as inputs.

The magic behind an XXE attack is that the XML specification includes a special annotation for importing external files. This special directive, called an external entity, is interpreted on the machine on which the XML file is evaluated. This means that a specially crafted XML payload sent to a server's XML parser could result in compromising files in that server's file structure.

- Often I consider these along the same lines as Java serialization attacks, sending data in JAVA that gets converted/expanded etc.

13. Injection



One of the most commonly known types of attacks against a web application is SQL injection. SQL injection is a type of injection attack that specifically targets SQL databases, allowing a malicious user to either provide their own parameters to an existing SQL query, or to escape an SQL query and provide their own query. Naturally, this typically results in a compromised database because of the escalated permissions the SQL interpreter is given by default.

SQL injection is the most common form of injection, **but not the only form**. Injection attacks have two major components: an interpreter and a payload from a user that is somehow read into the interpreter. This means that injection attacks can occur against command-line utilities like FFMPEG (a video compressor) as well as against databases (like the traditional SQL injection case).

- Injecting SQL commands, perform database actions
- Escape into the operating system via SQL, easier in the old days
- Escape into shell from URL parameters, often happens
- Compare to shellshock vuln, sending data that later ends up in the Unix shells

Sqlmap



sqlmap is an open source penetration testing tool that automates the process of detecting and exploiting SQL injection flaws and taking over of database servers. It comes with a powerful detection engine, many niche features for the ultimate penetration tester and a broad range of switches lasting from database fingerprinting, over data fetching from the database, to accessing the underlying file system and executing commands on the operating system via out-of-band connections.

Automatic SQL injection and database takeover tool http://sqlmap.org/

sqlmap features



'; Features();--

- Full support for **MySQL, Oracle, PostgreSQL, Microsoft SQL Server, Microsoft Access, IBM DB2, SQLite, Firebird, Sybase, SAP MaxDB and HSQLDB** database management systems.
- Full support for six SQL injection techniques: **boolean-based blind, time-based blind, error-based, UNION query-based, stacked queries and out-of-band**.
- Support to **directly connect to the database** without passing via a SQL injection, by providing DBMS credentials, IP address, port and database name.
- Support to enumerate **users, password hashes, privileges, roles, databases, tables and columns**.
- Automatic recognition of password hash formats and support for **cracking them using a dictionary-based attack**.
- Support to **dump database tables** entirely, a range of entries or specific columns as per user's choice. The user can also choose to dump only a range of characters from each column's entry.

Not a complete list!

Source: http://sqlmap.org/

Command injection flaws



Running operating system commands can often be done using *escaping*

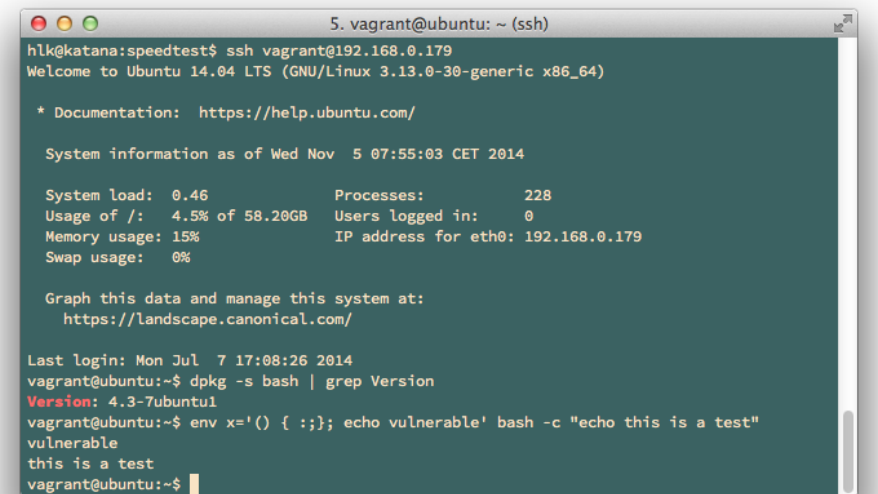
```
<pre>
<?php passthru(" netstat -an && ifconfig -a"); ?>
</pre>
```

Other shell escapes:

- Perl: `print `/usr/bin/finger $input{'command'}`;`
- UNIX shell: `` `echo hej` ``
- Microsoft SQL: `exec master..xp_cmdshell 'net user test testpass /ADD'`

May be exploited through web applications if data is not sanitized

Shellshock CVE-2014-6271 - and others



Can result in data being sent through several systems, before it reaches some shell program

Source: https://en.wikipedia.org/wiki/Shellshock_(software_bug)

November 2021: Log4Shell



It would not be possible to discuss 2021 in the context of vulnerabilities without the mention of Log 4Shell. **A widely used Java-based logging library caused headaches for Security professionals wor ldwide**. Many scrambled to quantify their use of Log4j within their estates.

A zero-day exploit quickly followed, confirming the worst - **Remote Code Execution (RCE) was inde ed possible.** However, what made the nature of the vulnerability even more challenging was the abilit y to exploit a backend logging system from an unaffected front end host. For example, an attacker can craft a weaponised log entry on a mobile app or webserver not running Log4j. The attacker could make their way through to backend middleware itself running Log4j, which significantly extends the attack surface of the vulnerability.

The NCSC even took the step of recommending the update was immediately applied, whether or not Log4Sh ell was known to be in use. As is commonly the case with critical vulnerabilities, two successive Log 4j patches were subsequently released in the week following the original addressing Denial of Service (DoS) and a further RCE. This further increased workloads of Security and IT teams just as they thou ght the worst of 2021 had been and gone.

Source - for this description:
https://chessict.co.uk/resources/blog/posts/2022/january/2021-top-security-vulnerabilities/

See also https://en.wikipedia.org/wiki/Log4Shell

14. Denial of Service (DoS)



Perhaps one of the most popular types of attacks, and the most widely publicized, is the distributed denial of service (DDoS) attack. This attack is a form of denial of service (DoS), in which a large network of devices flood a server with requests, slowing down the server or rendering it unusable for legitimate users.

DoS attacks come in many forms, from the well-known distributed version that involves thousands or more coordinated devices, to code-level DoS that affects a single user as a result of a faulty regex implementation, resulting in long times to validate a string of text. DoS attacks also range in seriousness from reducing an active server, to a functionless electric bill, to causing a user's web page to load slightly slower than usual or pausing their video midbuffer.

Because of this, it is very difficult to test for DoS attacks (in particular, the less severe ones). Most bug bounty programs outright ban DoS submissions to prevent bounty hunters from interfering with regular application usage.

- There have been some hash-based attacks and attacks using data to create bad performance, see PHP security advisories

15. Exploiting Third-Party Dependencies



The rampant use of third-party dependencies, in particular from the OSS realm, has created an easy-to-overlook gap in the security of many web applications. A hacker, bug bounty hunter, or penetration tester can take advantage of these integrations and jumpstart their search for live vulnerabilities. Third-party dependencies can be attacked a number of ways, from shoddy integrations to fourth-party code or just by finding known exploits discovered by other researchers or companies.

- Be careful what you include in your environment

CIS Controls: Offense informs defense



- **Offense informs defense:** Use knowledge of actual attacks that have compromised systems to provide the foundation to continually learn from these events to build effective, practical defenses. Include only those controls that can be shown to stop known real-world attacks.
- **Prioritization:** Invest first in Controls that will provide the greatest risk reduction and protection against the most dangerous threat actors and that can be feasibly implemented in your computing environment. The CIS Implementation Groups discussed below are a great place for organizations to start identifying relevant Sub-Controls.
- **Measurements and Metrics:** Establish common metrics to provide a shared language for executives, IT specialists, auditors, and security officials to measure the effectiveness of security measures within an organization so that required adjustments can be identified and implemented quickly.
- **Continuous diagnostics and mitigation:** Carry out continuous measurement to test and validate the effectiveness of current security measures and to help drive the priority of next steps.
- **Automation:** Automate defenses so that organizations can achieve reliable, scalable, and continuous measurements of their adherence to the Controls and related metrics.

Source: CIS-Controls-Version-7-1.pdf

Application Software Security



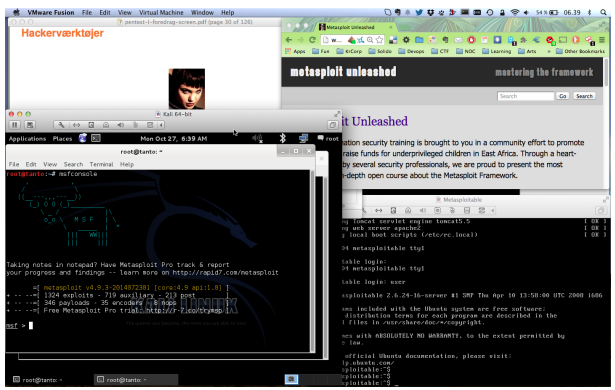
CIS Control 18:
Application Software Security
Manage the security life cycle of all in-house developed and acquired software in order to prevent, detect, and correct security weaknesses.

Source: Center for Internet Security CIS Controls 7.1 `CIS-Controls-Version-7-1.pdf`

CIS Control 16:
Application Software Security
Manage the security life cycle of in-house developed, hosted, or acquired software to prevent, detect, and remediate security weaknesses before they can impact the enterprise.

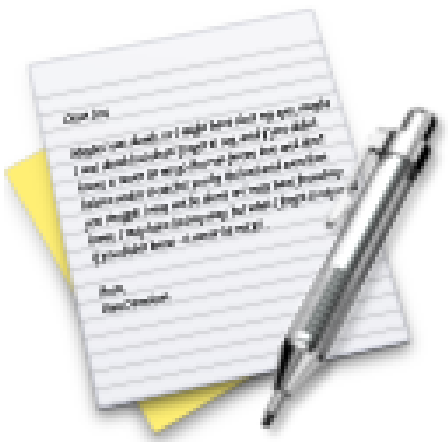
Source: Center for Internet Security CIS Controls v8 `CIS_Controls_v8_Guide.pdf`

Testing Labs



- Sniffers Wireshark and similar tools
- Proxies and fuzzers
- Debuggers
- Virtualisation - can also emulate ARM on Intel etc.
- Laptops and network hardware - dont use a HUB! Cheap managed switch with mirror port is better

Exercise



Now lets do the exercise

⚠ JuiceShop Attacks 60min

which is number 22 in the exercise PDF.

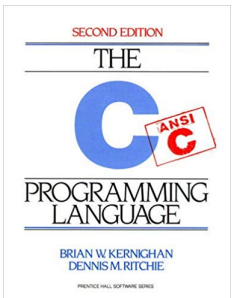
What is C



C is a general-purpose, procedural computer programming language supporting structured programming, lexical variable scope, and recursion, while a static type system prevents unintended operations. By design, C provides constructs that map efficiently to typical machine instructions and has found lasting use in applications previously coded in assembly language. Such applications include operating systems and various application software for computers, from supercomputers to embedded systems. **C was originally developed at Bell Labs by Dennis Ritchie between 1972 and 1973 to make utilities running on Unix.** Later, it was applied to re-implementing the kernel of the Unix operating system.[6] During the 1980s, C gradually gained popularity. **Nowadays, it is one of the most widely used programming languages**,[7][8] with C compilers from various vendors available for the majority of existing computer architectures and operating systems. C has been standardized by the ANSI since 1989 (see ANSI C) and by the International Organization for Standardization.

Source: https://en.wikipedia.org/wiki/C_(programming_language)

C language issues



- C was very portable
- C was standardized
- Still lots of unspecified behaviour / undefined behaviour and implementation-defined
- So C on one operating system can be very different!
- Writing portable and correct code is hard!

https://en.wikipedia.org/wiki/Unspecified_behavior

Hello World



> While small test programs have existed since the development of programmable computers, the tradition of using the phrase "Hello, World!" as a test message was influenced by an example program in the seminal 1978 book The C Programming Language.[3] The example program in that book prints "hello, world", and was inherited from a 1974 Bell Laboratories internal memorandum by Brian Kernighan, Programming in C: A Tutorial:[4]

```
main( ) {
        printf("hello, world\n");
}
```

- Very compact, easily readable - more than assembler anyway!

Code and quote from https://en.wikipedia.org/wiki/%22Hello,_World!%22_program

Hello World



```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
        (void) argc;
        (void) argv;

        printf("Hello world!\n");

        return EXIT_SUCCESS;
}
```

- Dont forget to define function return value, your variables and arguments to the program and functions
- Dont forget to return something useful
- Whats an int anyway? integers

Source: a better hello world from https://da.wikipedia.org/wiki/Hello_world-program#C

Data types



- Character types char, signed char, unsigned char
- Integer types, short int, int, long int, long long int - plus unsigned
- Floating types float, double, long double
- Bit fields a specific number of bits in an object
- Signed and unsigned!
- Byte order, little or big endian - named from 1726 novel Gulliver's Travels by Jonathan Swift
- Big endian also called network order, as used in TCP/IP suite
- Lots more types we wont discuss in all details

Data types have a *range of values*

See more specific at https://en.wikipedia.org/wiki/C_data_types
also interesting is the float format
https://en.wikipedia.org/wiki/Single-precision_floating-point_format

Example integer overflow



```c
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char **argv)
{       (void) argc;  (void) argv;
                short int i1 = 32767;
                printf("First debug int is %d\n", i1);
                i1++;
                printf("Second debug int is now %d \n", i1);
}
```

```
user@Projects:programs$ gcc -o int1 int1.c && ./int1
First debug int is 32767
Second debug int is now -32768
```

We wont do the match in binary!

Integer overflows



```
----[ 1.2 What is an integer overflow?
Since an integer is a fixed size (32 bits for the purposes of this paper),
there is a fixed maximum value it can store.  When an attempt is made to
store a value greater than this maximum value it is known as an integer
overflow.  The ISO C99 standard says that an integer overflow causes
"undefined behaviour", meaning that compilers conforming to the standard
may do anything they like from completely ignoring the overflow to aborting
the program.  Most compilers seem to ignore the overflow, resulting in an
unexpected or erroneous result being stored.

----[ 1.3 Why can they be dangerous?
Integer overflows cannot be detected after they have happened, so there is
not way for an application to tell if a result it has calculated previously
is in fact correct.  This can get dangerous if the calculation has to do
with the size of a buffer or how far into an array to index.  Of course
most integer overflows are not exploitable because memory is not being
directly overwritten, but sometimes they can lead to other classes of bugs,
frequently buffer overflows.  As well as this, integer overflows can be
difficult to spot, so even well audited code can spring surprises.
```

Source: *Basic Integer Overflows* by blexim http://www.phrack.com/issues.html?issue=60&id=10#article

Integer overflows



```
----[ 1.2 What is an integer overflow?
Since an integer is a fixed size (32 bits for the purposes of this paper),
there is a fixed maximum value it can store.  When an attempt is made to
store a value greater than this maximum value it is known as an integer
overflow.  The ISO C99 standard says that an integer overflow causes
"undefined behaviour", meaning that compilers conforming to the standard
may do anything they like from completely ignoring the overflow to aborting
the program.  Most compilers seem to ignore the overflow, resulting in an
unexpected or erroneous result being stored.

----[ 1.3 Why can they be dangerous?
Integer overflows cannot be detected after they have happened, so there is
not way for an application to tell if a result it has calculated previously
is in fact correct.  This can get dangerous if the calculation has to do
with the size of a buffer or how far into an array to index.  Of course
most integer overflows are not exploitable because memory is not being
directly overwritten, but sometimes they can lead to other classes of bugs,
frequently buffer overflows.  As well as this, integer overflows can be
difficult to spot, so even well audited code can spring surprises.
```

Source: *Basic Integer Overflows* by blexim http://www.phrack.com/issues.html?issue=60&id=10#article

Overiding types, type conversion



`Unsigned 00000001 is the same as 0000000000000001`

- Value-preserving - there is space in the new type to store all values
- Value-changing - new type cannot represent the same values, damn!
- To a wider type, we can *sign extend* for signed types and *zero extend* for unsigned types
- To a more narrow, need to truncate - bye bye bits, loose precision!
- Again, we wont repeat all the bit values and arithmetics, and how C extends the values
- Casting can be used to specify the explicit type conversion
  `(unsigned char) bob` - treat variable bob as an unsigned char type

Audit tip: The compiler can optimize out certain conversions! This can affect both cryptographic processes and other things! Beware

Type Conversion Vulnerabilities



```
int copy(char *dst, char *src, unsigned int len)
{
  while (len--)
    *dst++ = *src++
}
```

- Signed/unsigned conversions
- If you pass signed int to this, a conversion might result in a large value insted
- This large value will most likely result in overflow
- Most libc routines that take a size have `size_t` which is unsigned!
- Signed negative values become very large unsigned

Compiling C code



```
  user@Projects:~$ mkdir openssh ; cd openssh
  user@Projects:openssh$ wget https://cdn.openbsd.org/pub/OpenBSD/OpenSSH/portable/openssh-8.0p1.tar.gz
  2019-09-16 20:52:14 (5.13 MB/s) - ‘openssh-8.0p1.tar.gz’ saved [1597697/1597697]
  user@Projects:openssh$ wget https://cdn.openbsd.org/pub/OpenBSD/OpenSSH/portable/openssh-8.0p1.tar.gz.asc
    2019-09-16 20:52:18 (26.5 MB/s) - ‘openssh-8.0p1.tar.gz.asc’ saved [683/683]
  user@Projects:openssh$ gpg --verify openssh-8.0p1.tar.gz.asc
  gpg: assuming signed data in 'openssh-8.0p1.tar.gz'
  gpg: Signature made Thu 18 Apr 2019 12:54:07 AM CEST
  gpg:                using RSA key 59C2118ED206D927E667EBE3D3E5F56B6D920D30
  gpg: Can't check signature: No public key
// fix by getting key etc.
user@Projects:openssh$ tar zxf openssh-8.0p1.tar.gz
user@Projects:openssh$ cd openssh-8.0p1
user@Projects:openssh-8.0p1$ ./configure && make && sudo make install
```

- `./configure` checks compiler, headers, libraries like OpenSSL and zlib, system settings – writes the Makefile, stops if a library is missing, e.g. install `libssl-dev`
- Select features and paths, see `./configure --help`, e.g. `--prefix=/usr/local --with-pam`
- `sudo make install` runs the build scripts as root – verify the signature first!

Summary compiling software



- Very often Linux/Unix software was distributed as archives - tgz - tar gzipped
- Configure checks for libraries and such things, see GNU Autoconf – which options are required, which other libraries are available etc.
- Today most systems provide precompiled binaries as signed packages – and the distribution adds its own patches in the process
- Patches can fix vulnerabilities without changing the version number, a scanner reporting "old version" may be wrong – check with `apt changelog openssl`
- Patches often align the configuration with the operating system conventions, file locations, users, startup scripts
- Preferred to use binary package systems like Debian apt!
- Binary packages can be upgraded – and downgraded more easily, e.g. `apt install openssl=<version>`

Lets browse chapter 2 programming



Gray Hat Hacking: The Ethical Hacker's Handbook, 6th ed., Allen Harper and others, ISBN: 9781264268948 – examples show small programs, hello world style

- Introduces control structures, if/then/else, for loops, while loops
- Variables and arithmetics - including comparison
- Functions - in deep details with registers and stack pointers
- Talks about compilation and assembly - same process is done in interpreted languages before instructions run
- Strings are introduced, with types and pointers, hard concepts
- Format strings, how to print a string with printf
- The stack is described in detail, important for security/exploiting
- Above are the most important parts of the chapter, IMHO

Many books contain such material – use the one you have

Pointers



- Copy example already used pointers
- They point to a location in memory - a variable, a structure, some object of some kind
- Pointer arithmectic is dangerous
- ... but often used for reading structures, from the network
- Mismatched values, wrong asumptions, may result in pointers going wrong

Suricata VXLAN decode - defensive checks



```
int DecodeVXLAN(ThreadVars *tv, DecodeThreadVars *dtv, Packet *p,
        const uint8_t *pkt, uint32_t len, PacketQueue *pq)
{
    if (unlikely(!g_vxlan_enabled))
        return TM_ECODE_FAILED;
    if (len < (sizeof(VXLANHeader) + sizeof(EthernetHdr)))
        return TM_ECODE_FAILED;
    const VXLANHeader *vxlanh = (const VXLANHeader *)pkt;
    if ((vxlanh->flags[0] & 0x08) == 0 || vxlanh->res != 0) {
        return TM_ECODE_FAILED;
    }
#if DEBUG
    uint32_t vni = (vxlanh->vni[0] << 16) + (vxlanh->vni[1] << 8) + (vxlanh->vni[2]);
    SCLogDebug("VXLAN vni %u", vni);
#endif
    StatsIncr(tv, dtv->counter_vxlan);
```

Source: excerpt from https://github.com/OISF/suricata/blob/master/src/decode-vxlan.c
Dont read past end of buffer!

Structures



- Pointers and Structure padding
- Look how the structures for packets are defined

```
/* ARP stuff */
struct arp_data { /* ARP and Ethernet fields*/
    u_int16_t formathw;
    u_int16_t formatproto;
    u_int8_t lenhw;
    u_int8_t lenproto;
    u_int16_t op;
    /* Ethernet Data */
    u_int8_t mac_source[ETHER_ADDR_LEN];
    u_int8_t mac_dest[ETHER_ADDR_LEN];
};
```

Source: Yersinia 0.7 `src/arp.h`, a layer 2 attack tool

Look at the Wireshark 3.3 Packet Diagram



- Wireshark 3.3 has a packet diagram, which might be quite handy!
- Lets look into it, and if possible run a couple of network dumps
- Hopefully it will illustrate how network data is transferred
- Maybe install the newest version on your laptop
- Alternatively Lets look at C code from Suricata or Zeek, you choose

Lets browse the buffer overflow papers



- *Smashing The Stack For Fun And Profit*, Aleph One, Phrack 49, 1996 – stack based buffer overflow, shellcode, NOP sled

  https://phrack.org/issues/49/14
- *Bypassing non-executable-stack during exploitation using return-to-libc*, c0ntex, 2006 – return to `system("/bin/sh")` instead of shellcode

  https://www.exploit-db.com/papers/13204
- *Basic Integer Overflows*, blexim, Phrack 60, 2002 – widthness, arithmetic and signedness bugs

  https://phrack.org/issues/60/10

For each paper find: the vulnerable C code, what gets overwritten, and how execution is redirected

Papers from the 90s and 2000s – surely nobody writes code like that anymore. . . right?

Real vulns



> Hold tight, this may blow your mind. . . A low-privileged user account on most Linux operating systems with UID value anything greater than 2147483647 can execute any systemctl command unauthorizedly—thanks to a newly discovered vulnerability. The reported vulnerability actually resides in PolicyKit (also known as polkit)—an application-level toolkit for Unix-like operating systems that defines policies, handles system-wide privileges and provides a way for non-privileged processes to communicate with privileged ones, such as "sudo,"that does not grant root permission to an entire process. The issue, tracked as **CVE-2018-19788**, impacts PolicyKit version 0.115 which comes pre-installed on most popular Linux distributions, including Red Hat, Debian, Ubuntu, and CentOS. ... Where, `INT_MAX` is a constant in computer programming that defines what maximum value an integer variable can store, which equals to 2147483647 (in hexadecimal 0x7FFFFFFF).

- Reality bites again:
  https://thehackernews.com/2018/12/linux-user-privilege-policykit.html
- *Red Hat has recommended system administrators not to allow any negative UIDs or UIDs greater than 2147483646 in order to mitigate the issue until the patch is released.*

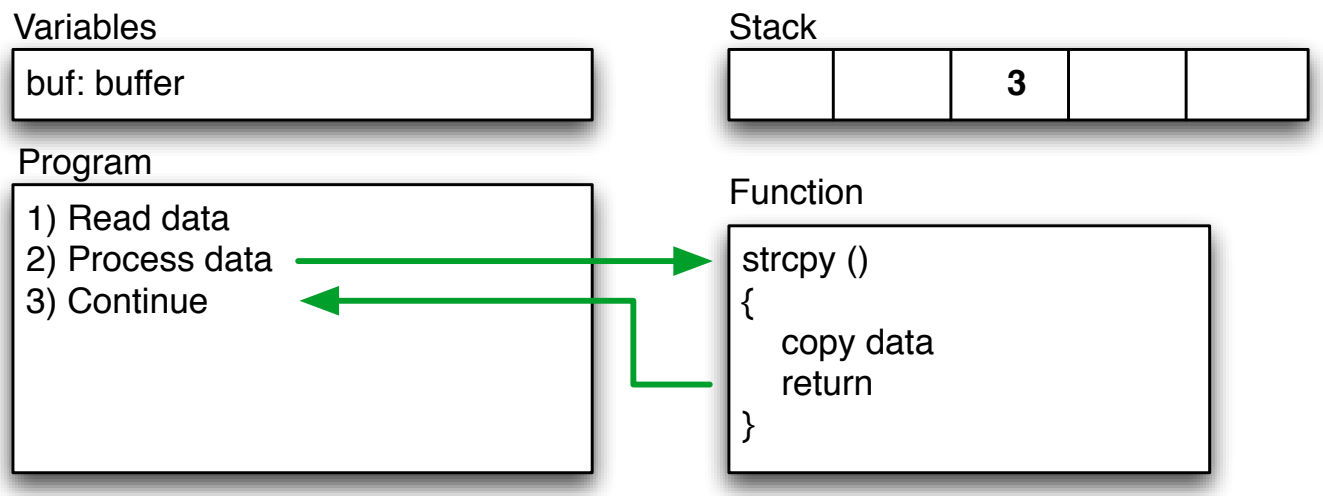
Buffer overflows a C problem



A buffer overflow is what happens when writing more data than allocated in some area of memory. Typically the program will crash, but under ce rtain circumstances an attacker can write structures allowing take over of return addresses, parameters for system calls or program execution.

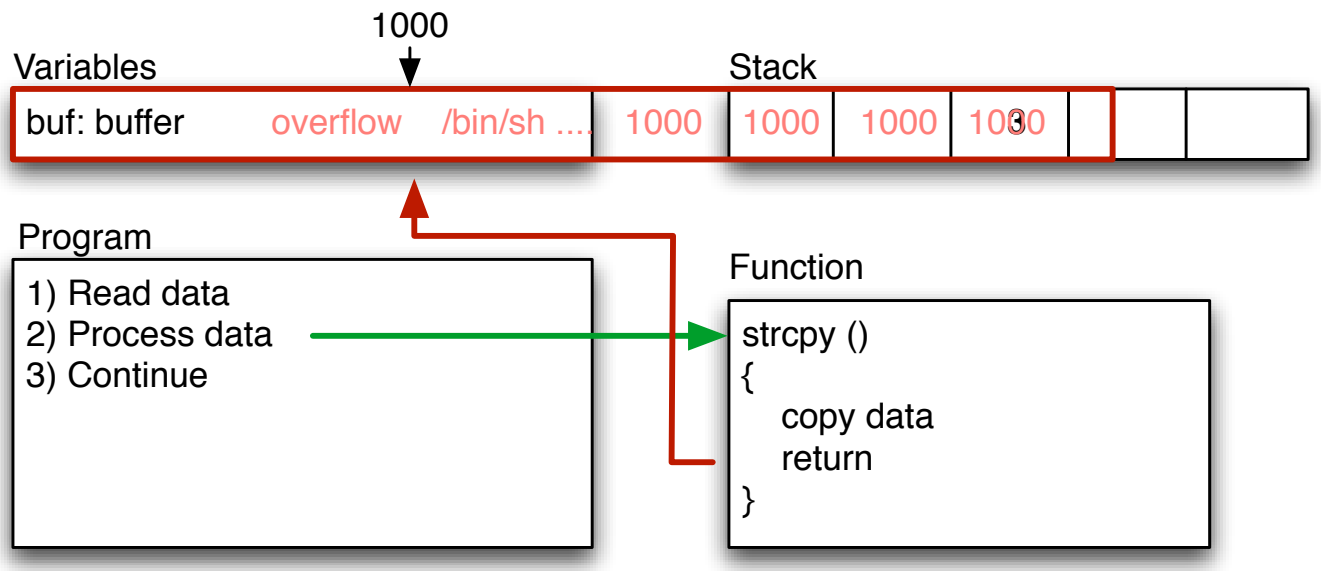
Stack protection is today used as a generic term for multiple technologies used in operating systems, libraries, compilers etc. that protect t he stack and other structures from being overwritten or changed through buffer overflows. StackGuard and Propolice are examples of this.

Buffers and stacks, simplified




```
main(int argc, char **argv)
{       char buf[200];
         strcpy(buf, argv[1]);
         printf("%s\n",buf);
}
```

Overflow – segmentation fault



- Bad function overwrites return value!
- Control return address
- Run shellcode from buffer, or from other place

Shell code



```
char *args[] = { "/bin/sh", NULL };

execve("/bin/sh", args, NULL);
```

- Shell code is the small payload an exploit runs after taking control – classically it spawns a shell with the `execve` system call above
- Written against system calls directly, so it needs no linker or libc – often hand-tuned assembly
- Shell code can be quite small, smallest seems to be 21 bytes for Linux x86
- Must avoid bytes that break the input path, e.g. a null byte stops `strcpy`
- A NOP sled (`0x90` . . .) in front lets an approximate return address still land in the payload
- You rarely write it yourself: generators like `msfvenom` produce it for a chosen platform

Defensive note: understanding shell code is how we reason about non-executable memory and other protections

Executable-space protection



Also most modern operating systems have various prevention mechanisms like ASLR, DEP, W xor X, NX, ...

- Executable space protection on Windows is called "Data Execution Prevention"(DEP). https://en.wikipedia.org/wiki/Executable-space_protection#Windows
- Mac OS X has no special brand name – it just uses the NX bit (W^X): NX stack from 10.4, NX stack+heap for 64-bit from 10.5
- `W^X` "write xor execute"W xor X https://en.wikipedia.org/wiki/W%5EX
- NX-bit no-execute https://en.wikipedia.org/wiki/NX_bit
- Address space layout randomization (ASLR) https://en.wikipedia.org/wiki/Address_space_layout_randomization

Return-to-libc



```
How does the technique look on the stack - a basic view will be something
similar to this:
[-] Buffer overflow smashing EIP and jumping forward to shellcode
                                                1                    2
|------------------|-----------|------------|--------------------------|
|             AAAAAAAAAAAA      |    RET     |        SHELLCODE         |
|------------------|-----------|------------|--------------------------|
                    args              EBP        EIP
[-] Buffer overflow doing return-to-libc and executing system function
                                                  1             2         3
|------------------------------|------------|--------------|------------|
|            buffer            |   system   |   fake_ret   |   /bin/sh  |
|------------------------------|------------|--------------|------------|
                    args              EBP        EIP
```

Instead of putting code on the stack, that cannot be executed, put on a fake return address which goes to a function in the C library, like `system("/bin/sh")`

Source: *Bypassing non-executable-stack during exploitation using return-to-libc* by c0ntex — c0ntex[at]gmail.com

Return-oriented programming (ROP)



To continue exploiting hacker have invented Return-oriented programming (ROP) which is one of the buzzing advanced exploitation techniques these days to bypass NX, ASLR

The main idea is to create a list of instructions from endings of functions, prepare a stack so when it returns into a function, it carries out some small part of the whole – a gadget.

Sources BlackHat
http://www.blackhat.com/html/bh-us-10/bh-us-10-archives.html

https://media.blackhat.com/bh-us-10/presentations/Zovi/BlackHat-USA-2010-DaiZovi-Return-Oriented-Exploitation-slides.pdf

Return Oriented Programming



- By doing *return chaining* build shell-code from existin program
- Instead of returning to functions, return to instruction sequences followed by a return instruction
- Can return into middle of existing instructions to simulate different instructions
- All we need are useable byte sequences anywhere in executable memory pages
- Scan executable memory regions of common shared libraries for useful instructions followed by return instructions

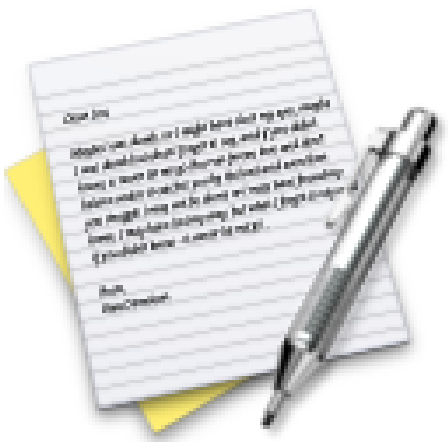
https://media.blackhat.com/bh-us-10/presentations/Zovi/BlackHat-USA-2010-DaiZovi-Return-Oriented-Exploitation-slides.pdf

Function call return



- Function calls are implemented using stacks, and they usually return
- By finding functions that do something small, and then return an exploit might be able to pack a stack, and return through an exploitation technique known as *Return Oriented Programming* (ROP)
- Note: there is a paper on the schedule Removing ROP Gadgets from OpenBSD Todd Mortimer mortimer@openbsd.org, can be downloaded at https://www.openbsd.org/papers/asiabsdcon2019-rop-paper.pdf
- This changes the rules, by changing the entry (prologue) and return from functions (epilogue): RETGUARD is a mechanism that adds instrumentation to the prologue and epilogue of each function that terminates in a return instruction.

Exercise

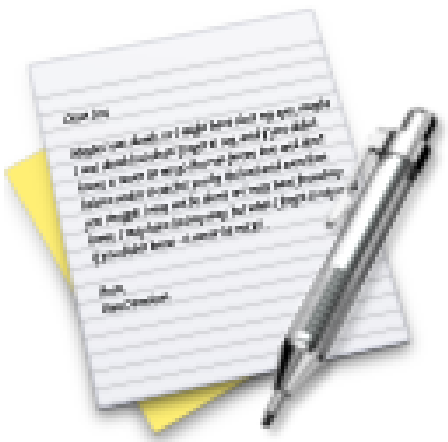


Now lets do the exercise

⚠ Small programs with data types 15min

which is number **26** in the exercise PDF.

Exercise

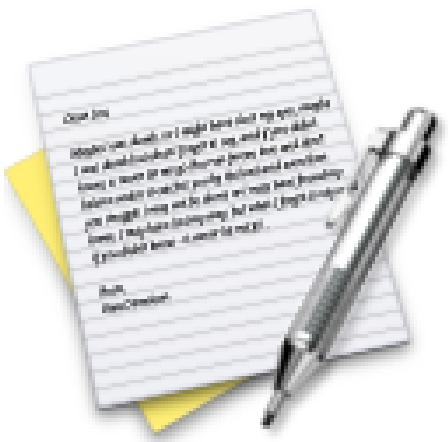


Now lets do the exercise

⚠ Pointers and Structure padding 30min

which is number 27 in the exercise PDF.

Exercise

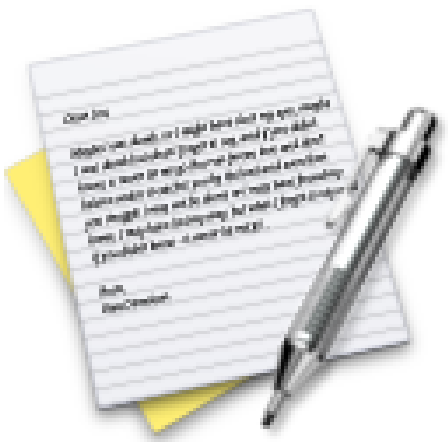


Now lets do the exercise

Wireshark 15 min

which is number **28** in the exercise PDF.

Exercise

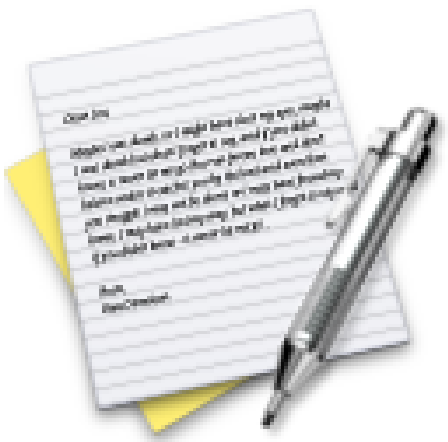


Now lets do the exercise

⚠ Trying PMD static analysis 30 min

which is number 29 in the exercise PDF.

Exercise

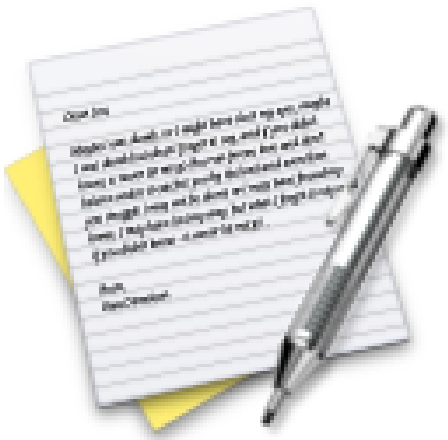


Now lets do the exercise

⚠ Git hook 30 min

which is number **30** in the exercise PDF.

Exercise

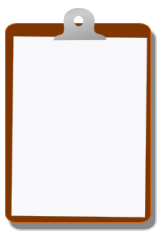


Now lets do the exercise

ℹ Use a XML library in Python up to 45min

which is number **31** in the exercise PDF.

For Next Time



Think about the subjects from this time, write down questions

Check the plan for chapters to read in the books

Visit web sites and download papers if needed

Retry the exercises to get more confident using the tools